

AI TRAINING – DAY 5 ASSIGNMENT

Python Loops: Repeat → Control → Break → Continue → Nest

Learning Guidance + Practical Problems + Student Deliverable

Level	Beginner → Medium
Suggested Time	3–4 hours
Core Topics	for, while, range(), break, continue, nested loops
Daily Deliverable	Loop Practice Notebook
Main Focus	Learn to think using repetition and loop control

Important: What Is the Student Deliverable?

This assignment is your learning guidance. You do not submit a separate file for every activity. Your final submission for Day 5 is ONE notebook called the Loop Practice Notebook. Put your practice, 10 problems, debugging work, real-world challenge and reflection inside that notebook.

Recommended filename: `Day_05_Loop_Practice_YourName.ipynb`

Learning Goals

- Understand for and while loops.
- Explain loops, range(), break, continue and nested loops in your own words.
- Use range() correctly.
- Understand when to use break and continue.
- Build nested loops.
- Identify common loop errors and improve your solution.
- Solve problems using loops instead of repeating code manually.
- Solve 10 loop problems including prime, palindrome and patterns.

How to Use This Assignment

1. Read and understand the concept before coding.
2. Predict output before running small examples.
3. Write your own solution before asking AI for help.
4. Run code with different inputs.
5. When code fails, identify and explain the error instead of only replacing the code.
6. Record final work in the Loop Practice Notebook.

PART A – Learn Before Coding

For every concept, explain it in your own words. Do not copy a definition.

1. for loop

When do I use a for loop?

A for loop is used when for loop is used execute the block of code repeatedly.

Write 2 simple examples.

Examples 1: `for i in range(1,4):`

```
    print(i)
```

output:1

2

3

Examples 2: `fruits = ["apple", "banana", "mango"]`

```
    for fruit in fruits:
```

```
        print(fruit)
```

output:apple

banana

mango

2. while loop

When do I use a while loop instead of a for loop?

A while loop is useful when while loop use when repetition depends on a condition.

Write 2 simple examples.

Examples 1: `count = 1`

```
    while count <= 5:
```

```
        print(count)
```

```
    count = count + 1
```

output:1

2

3

4

5

Examples 2: `count = 1`

```
    while count >=5:
```

```
        print(count)
```

```
    count = count - 1
```

output:5

4

3

2

1

3. range()

Experiment with:

```
range(5)
range(1, 6)
range(2, 11, 2)
range(10, 0, -1)

predict value :{0,1,2,3,4}

range:(5)

predict value:{1,2,3,4,5}

range:(1,6)

predict value:{2,4,6,8,10}

range:(2, 11, 2)

predict value:{10,9,8,7,6,5,4,3,2,1}

range:(10, 0, -1)
```

Predict the values before running. Understand:

```
range(start, stop, step)
```

The stop value is normally not included.

PART B – Loop Control

1. break

Create a loop that stops when a particular number is reached. Explain what happens when break executes.

```
Code:for i in range(1, 11):
    if i == 5:
        break
    print(i)
```

output:

1
2
3
4

2. continue

Create a loop that prints 1–20 but skips multiples of 3. Explain whether continue stops the entire loop.

```
code:for i in range(1,8):
    if i % 3 ==0:
```

```
        continue
    print(i)
```

Output:

```
1
2
4
5
7
```

3. Nested Loops

Create:

```
1 2 3
1 2 3
1 2 3
```

Identify the outer loop, inner loop and total inner-loop executions.

```
code:for i in range(3):
    for j in range(1, 4):
        print(j, end=" ")
    print()
```

output:

```
1 2 3
1 2 3
1 2 3
```

Outer loop: runs 3 times

it controls the 3 rows

Inner loop:run 3 times

it prints 1 2 3

Total inner -loop execution

Outer loop = 3 times

inner loop = 3 times

3*3=9 total inner - loops

PART C – Predict Before You Run

Predict first. Then run and compare.

```
for i in range(5):
    print(i)
```

Output:

```

0
1
2
3
4

for i in range(1, 6):
    if i == 3:
        break
    print(i)

```

Output:

```

1
2

for i in range(1, 6):
    if i == 3:
        continue
    print(i)

```

output:

```

1
2
4
5

i = 1
while i <= 5:
    print(i)
    i += 1

```

Output:

```

1
2
3
4
5

```

PART D – Solve 10 Loop Problems

For every problem record: Problem → My Approach → Code → Test Cases → Output → What I Learned.

Problem 1 – Multiplication Table

Ask for a number and print its table from 1 to 10. Focus: for + range().

```

num = int(input("Enter a number: "))

for i in range(1, 11):
    result = num * i

```

```
print(num, "x", i, "=", num * i)
```

Output:

Enter the number: 5

5 x 1 = 5

5 x 2 = 10

5 x 3 = 15

5 x 4 = 20

5 x 5 = 25

5 x 6 = 30

5 x 7 = 35

5 x 8 = 40

5 x 9 = 45

5 x 10 = 50

Problem 2 – Sum of Numbers

Ask for n and calculate $1 + 2 + \dots + n$. Focus: accumulator + for.

```
n = int(input("Enter a number: "))
```

```
total = 0
```

```
for i in range(1, n + 1):
```

```
    total = total + i
```

```
print("The total sum is:", total)
```

OUTPUT:

Enter a number: 5

The total sum is: 15

Problem 3 – Count Even and Odd Numbers

For 1 to n, count even and odd numbers. Focus: for + if.

```
n = int(input("Enter a number: "))
```

```
even_count = 0
```

```
odd_count = 0
```

```
for i in range(1, n + 1):
```

```
    if i % 2 == 0:
```

```
        even_count = even_count + 1
```

```
    else:
```

```
        odd_count = odd_count + 1
```

```
print("Even numbers:", even_count)
print("Odd numbers:", odd_count)
```

OUTPUT :

Enter a number: 10

Even numbers: 5

Odd numbers: 5

Problem 4 – Prime Number Checker

Determine whether a number is prime. First explain what makes a number prime.

```
num = int(input("Enter a number: "))
is_prime = True
if num <= 1:
    is_prime = False
else:
    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break
if is_prime == True:
    print(num, "is a prime number")
else:
    print(num, "is not a prime number")
```

OUTPUT :

Test 1:

Enter a number: 7

7 is a prime number

Test 2:

Enter a number: 10

10 is not a prime number

Problem 5 – Palindrome Number

Check whether a number such as 121 is a palindrome. Try without a ready-made palindrome function.

```
num = int(input("Enter a number: "))
original = num
reversed_num = 0
while num > 0:
    last_digit = num % 10
```

```

        reversed_num = (reversed_num * 10) + last_digit

    num = num // 10

if original == reversed_num:

    print(original, "is a palindrome")

else:

    print(original, "is not a palindrome")

```

OUTPUT:

Test 1:

Enter a number: 121

121 is a palindrome

Test 2:

Enter a number: 123

123 is not a palindrome

Problem 6 – Find Factors

Print all factors of a number. Example: 12 → 1, 2, 3, 4, 6, 12.

```

num = int(input("Enter a number: "))

print("Factors of", num, "are:")

for i in range(1, num + 1):

    if num % i == 0:

        print(i)

```

OUTPUT:

Enter a number: 12

Factors of 12 are:

1

2

3

4

6

12

Problem 7 – break Challenge

Search 1–100 and stop at the first number divisible by both 7 and 9. Use break.

```

print("Searching 1 to 100 for a number divisible by 7 and 9...")

for i in range(1, 101):

    if i % 7 == 0 and i % 9 == 0:

        print("Found it! The number is:", i)

```

```
break
```

OUTPUT:

Searching 1 to 100 for a number divisible by 7 and 9...

Found it! The number is: 63

Problem 8 – continue Challenge

Print 1–50 but skip numbers divisible by 3. Use continue.

```
print("Numbers from 1 to 50 (skipping multiples of 3):")
for i in range(1, 51):
    if i % 3 == 0:
        continue
    print(i, end=" ")
```

OUTPUT:

Numbers from 1 to 50 (skipping multiples of 3):

1 2 4 5 7 8 10 11 13 14 16 17 19 20 22 23 25 26 28 29 31 32 34 35 37 38 40 41 43 44
46 47 49 50

Problem 9 – Pattern Printing

Print *, **, ***, ****, ***** and the reverse. Bonus: centered pyramid.

```
rows = int(input("Enter number of rows: "))
print("\n--- Normal ---")
for i in range(1, rows + 1):
    print("*" * i)
print("\n--- Reverse ---")
for i in range(rows, 0, -1):
    print("*" * i)
print("\n--- Centered Pyramid (Bonus) ---")
for i in range(1, rows + 1):
    spaces = " " * (rows - i)
    stars = "*" * ((2 * i) - 1)
    print(spaces + stars)
```

OUTPUT:

Enter number of rows: 5

--- Normal ---

*

**

```

****

*****

--- Reverse ---

*****

****

***

**

*

--- Centered Pyramid (Bonus) ---

    *

   ***

  *****

 *****

*****

```

Problem 10 – Number Pattern

Print 1, 12, 123, 1234, 12345 and then 1, 22, 333, 4444, 55555. Use nested loops.

```

rows = int(input("Enter number of rows: "))

print("--- Pattern 1 ---")

for i in range(1, rows + 1):
    for j in range(1, i + 1):
        print(j, end="")

    print() # Moves to the next line

print("--- Pattern 2 ---")

for i in range(1, rows + 1):
    for j in range(1, i + 1):
        print(i, end="")

    print() # Moves to the next line

```

OUTPUT:

Enter number of rows: 5

--- Pattern 1 ---

```

1
12
123
1234
12345

```

--- Pattern 2 ---

1

22

333

4444

55555

PART E – Debugging Lab

For every bug: identify the problem, explain why it happens, fix it and test it.

```
Bug 1
for i in range(5)
    print(i)
```

Problem: In for loop definition miss the ":"(colon) after range(5)

Correct code:

```
for i in range(5):
    print(i)
```

output:

0 1 2 3 4

What is missing?

The Colon: is missing

```
Bug 2
i = 1
while i <= 5:
    print(i)
```

Problem: Didn't write the increment/ decrement statement.

Correct code:

```
i = 1
while i <= 5:
```

```
    print(i)
```

```
    i+=1
```

output: 1 2 3 4 5

Why can this become an infinite loop? Fix it.

It can become infinite loop because i is never increased /decreased. Therefore i remains 1, and the condition is always true.

```
Bug 3
for i in range(1, 10):
    if i == 5:
        break
    print(i)
print('Loop completed')
```

```
code:for i in range(1, 10):
    if i == 5:
        break
    print(i)
    print('Loop completed')
```

Output:

```
1
Loop completed
2
Loop completed
3
Loop completed
4
loop completed
```

Understand where the loop actually stops..

The loop stops when the i reaches the value 5 .The break statement stop the loop and immediately comes out.

```
Bug 4
for i in range(1, 10):
    if i % 2 == 0:
        continue
    print(i)

code:for i in range(1, 10):
    if i % 2 == 0:
        continue
    print(i)
```

Output:

```
1 3 5 7 9
```

Why will print(i) never execute for even numbers?

Because we write if condition when i value divide with 2 and the remainder becomes zero the continue statement stop the current iteration and moves to next iteration.

```
Bug 5
for i in range(3):
    for j in range(3):
        print('*')

code:for i in range(3):
    for j in range(3):
        print('*')
```

Output:***

Predict how many times * prints, then verify.

For each outer loop execution, the inner loop executes 3 times.

Therefore $3 \times 3 = 9$

so * prints 9 times.

PART F – Real-World Challenge

Choose ONE: ATM transaction loop; Login system with 3 attempts; Shopping cart total calculator; Student marks analyzer; Number guessing game.

- Use a loop.
- Use at least one if.
- Use at least one break OR continue.
- Take user input.
- Display proper output.
- Test at least 3 different cases.

```
correct_name="lasya"
correct_password="@123"

for attempt in range(3):
    name = input("Enter name:")
    password = input("Enter password:")

    if name == correct_name and password == correct_password:
        print("Login successful!")
        break
    else:
        print("incorrect name or password.")

else:
    print("You have used all 3 attempts.Account locked. ")

Output:
```

Test case 1:

Enter name: lasy

Enter password: 123

Incorrect name or password.

Test case 2:

Enter name: lasya

Enter password: 123

Incorrect name or password.

Test case 3:

Enter name: lasya

Enter password: @123

Login successful!

PART G – Explain in Your Own Words

1. What is a loop?

Ans: loop is used to repeat a block of code multiple times

2. What is the difference between for and while?

Ans: for loop is used to execute the block of code repeatedly.

while loop is used to repeat a block of code as long as a condition is true

3. What does range() do?

Ans: range() gives a sequence of numbers for a loop

4. What happens when break executes?

Ans: break stops the loop completely

5. What happens when continue executes?

Ans: continue is used to move to the next one

6. What is a nested loop?

Ans: a nested loop is a loop inside another loop

7. When can a while loop become infinite?

Ans: A while loop can become infinite if its condition never becomes false

8. Why is indentation important inside loops?

Ans: Indentation is important because it tells Python which statements belong inside the loop."

9. What was the hardest problem today?

Ans: The number pattern problem, because tracking two loops at once is confusing.

10. What mistake did you make and how did you fix it?

Ans: I made a spacing mistake in Bug 4. I fixed it by moving the print statement to the left.

PART H – Self Check

Skill	Comfortable	Need Practice
for loop	☒	☐
while loop	☒	☐
range()	☒	☐
break	☒	☐
continue	☒	☐
Nested loops	☒	☐
Prime number	☒	☐

Palindrome	☐	☒
Pattern printing	☐	☒
Debugging loops	☒	☐

STUDENT DELIVERABLE – WHAT YOU MUST SUBMIT

Submit ONE notebook only. Everything below should be inside your Loop Practice Notebook.

`Day_05_Loop_Practice_YourName.ipynb`

Your notebook must contain:

1. Concept Practice – your own examples for for, while, range(), break, continue and nested loops.
2. 10 Loop Problems – complete solutions for all 10 problems.
3. Debugging – corrected versions and explanations.
4. Real-World Challenge – one completed loop-based project.
5. Reflection – answers to the 10 explanation questions.
6. Test Evidence – at least 2–3 test cases for important programs.

You DO NOT need to submit:

- A separate file for each problem.
- A separate debugging file.
- A separate notes file.
- A separate Word or PDF document.
- Screenshots for every program.

Simple Rule

**LEARN → PRACTICE → SOLVE 10 PROBLEMS → DEBUG → BUILD → REFLECT
→ SUBMIT ONE NOTEBOOK**

Submission Checklist

- ☒ Concept practice completed.
- ☒ All 10 loop problems completed.
- ☒ Prime number completed.
- ☒ Palindrome completed.
- ☒ Pattern problems completed.
- ☒ Debugging completed.
- ☒ Real-world challenge completed.

- ☒ Reflection completed.
- ☒ Test cases included.
- ☒ Notebook filename follows the required format.
- ☒ Final notebook is ready to submit.

DAY 5 PRINCIPLE

Don't just repeat code. Learn to control repetition.